

Project Guidelines (2024/2025)

Intro

Your final project is your opportunity to dive deep into a problem at the frontier of Machine Learning. You are encouraged to use powerful tools like large language models (GPT o3, Gemini 2.5 Pro, Claude, etc.) to refine your ideas, explore open research directions, draft proposals, and accelerate your technical thinking. Start by identifying a compelling question, study the foundational and recent papers in that area, and don't be afraid to be bold -- many impactful ideas start as creative hypotheses!

These projects are more than exercises; several from previous editions of this course have evolved into publications at top-tier conferences such as NeurIPS, ICML, ICLR, and CVPR. Think of this as a real research endeavor, not just a class assignment.

Examples of past student projects that became full publications:

- MERGE3: Efficient Evolutionary Merging on Consumer-grade GPUs, **ICML 2025**
- Task Singular Vectors: Reducing Task Interference in Model Merging, **CVPR 2025**
- COCOLA: Coherence-Oriented Contrastive Learning of Musical Audio Representations, **ICASSP 2025**
- MASS: MoErging through Adaptive Subspace Selection, **ICCV 2025** (*under review*)
- STAGE: Stemmed Accompaniment Generation through Prefix-Based Conditioning, **ISMIR 2025** (*under review*)
- LoopGen: Training-Free Loopable Music Generation, **ISMIR 2025** (*under review*)
- Activation Patching for Interpretable Steering in Music Generation, **ISMIR 2025** (*under review*)
- ATM: Improving Model Merging by Alternating Tuning and Merging, **ECAI 2025** (*under review*)

Evaluation Criteria (Total: 30 points)

- **Problem clarity & motivation (5 pts):** Is the problem well defined and relevant? Is the motivation grounded in literature or application need?
- **Literature review (5 pts):** Have key papers been cited and understood? Is the novelty of the approach contextualized clearly?
- **Technical depth (10 pts):** Is there a thoughtful method or architecture proposed and implemented? Are experiments well designed?
- **Creativity & ambition (5 pts):** Does the project show originality and an exploratory mindset?
- **Clarity & communication (5 pts):** Is the final report or presentation clear, well structured, and easy to follow?

Before you start

- Fill up [this google sheet](#) with your **matr. number** and the **project id** of your choice. If you do a team project, use a coherent **background color** for your matr. number, the same for all team members. Ignore the **title** column. Each team should use its own color.
- **Team projects** must be approved beforehand, with clear motivations and separation of tasks.
- If you want to propose your own project, see **project ID 0** below. If approved, add the project to the google sheet with project id 0, and write a short title in the **title** column.
- Projects that are not registered in the google sheet **will not be graded**.

What to deliver

Each project submission must consist of:

- A **report** describing the project and its outcomes, adhering to the **provided template**. **Check the page limits** as described in the template.
- A GitHub (or similar) **repository** with code, data, notebooks, checkpoints, results, and any other supplementary material.

Send links and pdf to rodola@di.uniroma1.it and solombrino@di.uniroma1.it with the subject line starting with **[ML Project]**.

When to deliver

There is no fixed date, you can deliver the project anytime **until May 31st, 2026**. After that date, you must pick a new project from the new project pool.

Remember to **register on Infostud** for the exam session when you want your project to be graded.

ID 0: Define your own project

You are free to define and pursue your own research idea in the domain of machine learning, on your favorite topic! Your goal is to explore a problem that genuinely interests you, apply state-of-the-art techniques (or invent your own), and aim for originality and insight.

Requirements:

- You must submit a **1-page proposal** describing your idea, motivation, related work, technical approach, and expected outcomes.
- The proposal must be approved before you begin. Submit it as early as possible.
- Creativity, ambition, and relevance to current research are strongly encouraged.
- If appropriate, you may work with a public dataset or bring your own.

This is your chance to shape the final project around your own research goals or curiosities. Some of the best course projects (and eventual publications) have come from this track.

ID 00: Projects from the Deep Learning and Applied AI course (Advanced)

You are free to pick a project from the [DLAI list of projects](#). These projects will be granted **extra points** due to the added difficulty of dealing with topics and methodology that were not covered in the course!

If you choose one of these projects, please **prepend 00** to the project id in the registration sheet. For example, project id 9 from the DLAI list should be denoted as ID **009** in the google sheet.

ID 1: AWOL for Audio — Language-to-Sound Generation via Parametric Synthesis

[AWOL \(Analysis WithOut synthesis using Language\)](#) is a method for generating 3D shapes (rigged animals and procedural trees) by learning a mapping from CLIP latent space to parametric model parameters. It bypasses traditional differentiable renderers by directly predicting meaningful model parameters from text or image embeddings. The method shows that, with a small set of labeled shape-text pairs, it's possible to generalize and interpolate in shape space using language, thanks to the structure of CLIP's joint image-text latent space and a Real-NVP flow model with learned binary masks.

This project explores **translating the AWOL framework from 3D geometry to another modality of your choice**. It might be audio, video, pixel art, you name it!

In brief, instead of predicting parameters for tree or animal shape models, you'll aim to **generate other data** (e.g., musical instruments, speech, new Pokemon monsters, etc.) from language by learning a mapping from a multi-modal model (e.g. CLAP) to the parameter space of an external generator (e.g. a sound synthesizer, or a pixel art drawing app).

A possible line of attack is to replicate AWOL's structure:

- **Embed language** into a latent space (e.g., CLAP);
- **Map it to synthesis parameters** via a model like Real-NVP with learned masking;
- **Render** the final output using the synthesizer (replacing Blender or SMAL+).

You are expected to read and understand the AWOL paper, especially how CLIP embeddings and the flow model are used to predict shape parameters, and ultimately build a parametric generator with well-defined control variables (preferably ones that can produce structured or semantic variation). Ideally, you want to explore extrapolation or interpolation in semantic space (e.g., generating data like sounds and pixel art for unseen textual prompts).

This project is open-ended. Success could mean anything from new types of musical instrument control, to realistic art generation, to **insights about grounding language in new modalities**.

ID 2: Audio Restoration for Generative Models — Improving MusicGen Outputs

Recent generative audio models like [MusicGen](#) produce impressive musical samples directly from text. However, these generations often suffer from **low bit depth, quantization artifacts, noise**, and a general lack of high-fidelity detail compared to studio-quality audio. In this project, your task is to **design a restoration pipeline** to enhance the quality of audio generated by such models. The goal is not to improve *semantic accuracy* (e.g. prompt adherence), but to **elevate perceptual and technical audio quality**.

You are free to approach the problem however you like:

- Post-process generated audio with learned models (e.g. diffusion, neural upsamplers, speech denoisers).
- Fine-tune MusicGen (or any other model) to favor higher fidelity outputs, if feasible.
- Use techniques from audio super-resolution, bandwidth extension, source separation, or even hybrid DSP+ML approaches.
- Work fully zero-shot using existing pretrained models, or dive into finetuning paradigms if compute permits.

Key challenges you might address include reducing quantization noise or compression artifacts, enhancing timbral clarity and spatial realism, increasing dynamic range or frequency resolution, or matching enhanced audio to ground truth samples if available.

Creativity and rigorous evaluation are key. You may use human listening tests, spectral analysis, CLIP-style perceptual similarity, or any other metric you justify.

ID 3: Multi-Model Merging via Spherical Barycenters (SLERP for $n>2$)

Model merging techniques have recently emerged as powerful tools for interpolating between two neural network checkpoints while preserving meaningful behavior (see [SLERP primer blog post](#)). The core idea is to treat model weights as high-dimensional vectors and interpolate them on the unit hypersphere, rather than in Euclidean space. This avoids destructive interference and yields smoother transitions in model capabilities.

In this project, you will **extend SLERP to the case of $n>2$ models** by formulating and implementing a **spherical barycenter** approach: effectively a generalization of SLERP that finds a weighted average of multiple models on the hypersphere. Additionally, you will explore how to ensure **cycle-consistent merging**, meaning that merging along different paths in model space yields consistent results.

Key objectives include:

- Implement **spherical barycenter averaging** of n model checkpoints, using appropriate optimization on the hypersphere.
- Test the merged model's performance on a downstream task (e.g., image classification, language modeling) and compare it with baselines like pairwise SLERP, linear interpolation, and naive ensembling.
- Optionally explore geometric interpretations of the merge space, such as convex hulls on the sphere of models.

You are free to pick your model family (e.g., simple MLP, logistic regression, etc.) and application domain. The main challenge is mathematical: merging networks meaningfully in a high-dimensional, non-Euclidean space.

ID 4: Orthogonal Re-Basin — Beyond Permutations in Model Merging

The [Git Re-Basin framework](#) shows that independently trained models with the same architecture can often be **aligned via permutation of hidden units**, allowing seamless merging and linear mode connectivity. However, this technique is strictly limited to **permutation matrices**, which form only a small subgroup of the full orthogonal group.

In this project, you will **relax the permutation constraint** and explore **general orthogonal transformations** as a means to align and merge models. Instead of aligning hidden units strictly by reordering (as in the Git Re-Basin method), you'll investigate whether orthogonal matrices (e.g. rotations or reflections) can be applied layerwise to align internal representations, even if this introduces minor incompatibilities with nonlinearities.

Your goals:

- Develop a method to compute optimal orthogonal transformations (e.g. via **Procrustes** analysis, spectral decomposition, or optimization on the Stiefel manifold) that align hidden activations or weights across models.
- Quantify and analyze the **residual misalignment error** due to non-commutativity with nonlinearities like ReLU.
- Evaluate **cycle-consistency** and **interpolation behavior** in weight space (e.g., zero-barrier linear or spherical interpolation between orthogonally-aligned models).
- Compare your method with the original Git Re-Basin permutations and SLERP model merging.

This project combines **deep learning theory**, **manifold optimization**, and **geometry-aware model merging**. It opens the door to broader symmetry-aware model composition strategies beyond discrete permutations.

ID 5: Block-Floating-Point Audio Representation for Efficient Learning and Inference

In the early 1990s, Amiga demo coders used a clever trick to boost the effective resolution of 8-bit audio hardware: by dynamically adjusting the **volume register** per sample, they simulated **14-bit playback** using only 8-bit samples + 6-bit volume. This system, known as the **DYN format**, functioned as a form of **block-floating-point encoding** where each sample carried coarse and fine detail via two small integers.

This project will **revive and modernize the DYN technique** in the context of **deep learning for audio**. You'll investigate whether splitting audio into a mantissa (8-bit) and a gain signal (6-bit) can serve as a **compact, trainable representation** for:

- waveform autoencoders or diffusion vocoders,
- low-bitrate generative models,
- robust input encoding for edge devices with quantized compute.

The main objectives can be summarized as follows:

1. **Implement DYN encoding**: Take 16-bit audio and convert it to an “8-bit sample + 6-bit gain” format using volume companding.
2. **Design a model** (e.g. convolutional autoencoder, diffusion vocoder) that consumes this split representation:
 - Treat sample and gain as separate input streams.
 - Fuse them via a learned or fixed multiply in the output layer.
3. **Compare** with baseline models trained directly on 16-bit PCM or 8-bit μ -law.
4. **Evaluate** performance using MOS, PESQ, STOI, and compression ratio.

Since this is a highly technical project, **you are free to change the above pipeline** and propose your own, or **modify the project objectives** to something that you deem more feasible or impactful.

As optional modifications / extensions you might consider replacing the deterministic gain with a learnable front-end (e.g., learnable compander or LEAF-like module), use the gain stream as a **side-channel condition for generation** (e.g., energy contour guidance), or quantize both streams and assess impact on training/inference cost.

Feel free to experiment!

This approach offers a principled way to trade **bit-depth for dynamic range**, with strong connections to floating-point quantization in hardware (e.g. FP8 training), mixed-precision DSP, and **audio representation learning under resource constraints**.